# FinWise AI - Hackathon Submission Document

## Problem Statement & Solution Statement

PROBLEM STATEMENT:
College students and young adults globally face a severe crisis of practical financial literacy. Entering the adult world, they are suddenly burdened with managing educational loans, fluctuating incomes, and living expenses without any formal financial education. This lack of knowledge frequently leads to compounding credit card debt, poor budgeting habits, and an inability to understand the long-term impact of compound interest. Traditional banking applications fail to address this issue. They are overwhelmingly complex, intimidating, and primarily retroactive-they only show a student what they have already spent, offering no proactive guidance on what they should do next. Furthermore, standard financial tools operate in a vacuum; they provide raw numbers but fail to deliver hyper-personalized, contextual advice based on a user's unique income, savings goals, and debt realities.

SOLUTION STATEMENT:
FinWise AI directly solves this crisis by serving as an intelligent, gamified financial copilot tailored specifically for the student demographic. Rather than simply logging numbers, FinWise AI abstracts complex financial mathematics-such as Amortization schedules, compound interest, and debt-to-income affordability ratios-into an intuitive, highly visual dashboard.

To guarantee safety, we enforce a strict 'Financial Safety Principle.' Our FastAPI backend utilizes ACID-compliant PostgreSQL transactions to perform all authoritative calculations, ensuring mathematical accuracy and absolute security that cannot be manipulated on the client side. We combine proactive tools, such as dynamic category-based budgets and gamified savings goals, with a powerful AI Copilot. Instead of offering generic, one-size-fits-all advice, our platform serializes the user's exact database metrics-including their active loan EMIs, wallet balance, and a proprietary 100-point Financial Health Score-and feeds them directly into our AI engine. This allows FinWise AI to deliver mathematically precise, hyper-personalized financial guidance. By transforming overwhelming banking data into actionable, educational insights, we empower students to build sustainable wealth and navigate their financial futures with confidence.

# FinWise AI - Hackathon Submission Document

## Technology Used - IBM Bob

IBM Bob was utilized as the central cognitive engine and development accelerator for the FinWise AI platform, fundamentally shaping both our software architecture and the core user experience. Its integration spans across direct user-facing features, complex algorithmic generation, security enforcement, and workflow optimization, allowing us to build a highly sophisticated, enterprise-grade application at hackathon speed.

1. The AI Copilot (Core Feature Integration & Context Injection):
The most prominent and innovative implementation of IBM Bob is within the platform's 'AI Copilot' feature (housed in AIBotPage.tsx and ai.py). Standard financial applications provide raw numbers without context; IBM Bob acts as the contextual bridge that translates raw data into human-readable strategy. When a student asks a question like 'Why is my financial health score failing?' or 'Can I afford to take out a new loan?', the FastAPI backend intercepts this request. Instead of blindly passing the prompt to the LLM, our backend securely serializes the user's actual PostgreSQL database records. It compiles their proprietary 6-axis Financial Health Score, exact wallet balance, active budget deficits, and upcoming EMI obligations into a strict JSON payload.

This payload is then injected into the System Prompt of the IBM Bob LLM endpoint using a Retrieval-Augmented Generation (RAG) inspired approach. Because IBM Bob processes this highly contextual prompt, it does not hallucinate generic advice. Instead, it generates hyper-personalized, mathematically accurate directives. For example, IBM Bob will analyze the data and respond with, 'You currently have an EMI-to-income ratio of 40%. You must reduce your Food budget by $30 this week and delay your Laptop savings goal to comfortably afford your upcoming loan payment.' By leveraging IBM Bob, we transformed static database rows into an interactive, educational dialogue that actively mentors the student.

2. Algorithmic Generation and Edge-Case Validation:
Behind the scenes, IBM Bob technology was heavily utilized during the development phase to architect, refine, and validate our complex financial algorithms. Developing accurate, edge-case resilient formulas for reducing-balance EMIs and compound interest calculations required immense precision to ensure they matched real-world banking standards. Furthermore, we designed a proprietary 100-point Financial Health Score that calculates weighted averages across six different financial sectors (Savings Rate, Budget Adherence, Debt Burden, Spending Stability, Goal Progress, and Emergency Reserves).

IBM Bob served as our mathematical co-pilot, assisting in generating the core Python models in our finance.py service. Critically, IBM Bob helped us identify and account for deep programmatic edge cases, such as the IEEE 754 'Banker's Rounding' standard inherent in Python 3. (For instance, ensuring that a neutral default score of 61.25 mathematically rounds to an exact 61.2 for new users, preventing floating-point UI errors). By utilizing IBM Bob for algorithm validation, we ensured our backend logic remained enterprise-grade, mathematically sound, and secure against overflow errors.

3. Security Hardening and Schema Validation:

# FinWise AI - Hackathon Submission Document

Financial applications require unparalleled security. We leveraged IBM Bob to harden our application against injection attacks and malformed data. On the frontend, IBM Bob assisted in generating strict, comprehensive Zod validation schemas for our React Hook Forms, ensuring that users cannot input negative values for expenses or bypass character limits. On the backend, it helped structure our Pydantic models in FastAPI to rigorously sanitize all incoming API payloads before they touch the SQLAlchemy ORM. This AI-assisted security pipeline ensures that the ACID-compliant PostgreSQL database remains pristine and impenetrable.

4. DevOps, Infrastructure, and Workflow Optimization:

Finally, we integrated IBM Bob into our development workflow to streamline the decoupling of our architecture and handle DevOps orchestration. By acting as a technical advisor, IBM Bob guided the strict separation of concerns within our repository. It ensured the React frontend strictly handled UI caching (via TanStack React Query) and global state (via Zustand), while the FastAPI backend exclusively handled token generation and business logic. Additionally, IBM Bob assisted in drafting our docker-compose.yml files, orchestrating the seamless containerization of our Nginx frontend, Python backend, and PostgreSQL database. This allowed our team to focus entirely on feature development, building a robust, scalable product at record speed without compromising on infrastructure quality.